

Theory of Computation (BCSE0404X(TOC))

Module: 3

Context Free Grammar and Push Down Automata

Course Details (B.Tech CSE 4th Sem)

Vivek Kumar Sharma
Assistant Professor
(CSE Department)



Faculty Profile

Evaluation Scheme

**NOIDA INSTITUTE OF ENGINEERING & TECHNOLOGY, GREATER NOIDA, GAUTAM BUDDH NAGAR
(AN AUTONOMOUS INSTITUTE)**

**Bachelor of Technology
Information Technology**

**Evaluation Scheme
SEMESTER-IV**

Sl. No.	Subject Codes	Subject	Types of Subjects	Periods			Evaluation Schemes				End Semester		Total	Credit
				L	T	P	CT	TA	TOTAL	PS	TE	PE		
1	BASCC0401	Employability Skill Development - II	Mandatory	2	0	0	60	40	100				100	2
2	BCSE0402	Database Management Systems	Mandatory	3	0	0	30	20	50		100		150	3
3	BCSE0401	Data Structures and Algorithms-II	Mandatory	3	0	0	30	20	50		100		150	3
4	BCSE0404X	Theory of Computation	Mandatory	3	0	0	30	20	50		100		150	3
5		Department Elective - I	Departmental Elective	3	0	0	30	20	50		100		150	3
6	BAS0403N	Statistics and Probability	Mandatory	3	1	0	30	20	50		100		150	4
7	BCSE0452Z	Database Management Systems Lab	Mandatory	0	0	4				50		50	100	2
8	BCSE0451	Data Structures and Algorithms-II Lab	Mandatory	0	0	2				25		25	50	1
9	BCSE0455	Web Technologies	Mandatory	0	0	6				50		100	150	3
10	BCSE0459	Mini Project	Mandatory	0	0	2				50			50	1
11	BCSCC0452	Problem Solving Approaches	Mandatory	0	0	2				50			50	1
12	BNC0402Y/ BNC0401Y	Environmental Science / Artificial Intelligence and Cyber Ethics	Compulsory Audit	2	0	0	30	20	50				50	NA
		*Massive Open Online Courses (For B.Tech. Hons. Degree)	*MOOCs											
		Applied English	VAC	1	0	0								
		TOTAL		20	1	16			350	225	500	175	1250	26

Syllabus

Course Contents / Syllabus		
Module 1	Introduction to Finite Automata and Compiler	10 hours
Introduction: Role of Automata and Formal languages, Alphabet, String, Grammar, Language, Chomsky Hierarchy of languages. Introduction to Finite State Machine: Deterministic Finite Automaton (DFA) and Non-Deterministic Finite Automaton (NFA), NFA with ϵ -Transition, Equivalence of NFA and DFA, Introduction to Compiler: Translators, Language Processing System, Phases and passes of compilation Use Case 1 : Role of Finite Automata in NLP and Speech Recognition.		
Module 2	Regular Expression and Tokenization	9 hours
Regular Expression: Regular Expression, Regular Sets, Properties of Regular Expression, Finite Automata and Regular Expression, Arden's Theorem, Regular Grammars-Right Linear and Left Linear grammars. Lexical Analyzer: Role of lexical Analyzer, Specifications and Recognition of tokens, Lex,		
Module 3	Context free grammar and Push Down Automata	09 hours
Context Free Grammar (CFG): Definition and Language, Derivations, Parse Trees and Ambiguity, Simplification of CFG, Push Down Automata: Definition of the Pushdown Automata, Languages of PDA		
Module 4	Parser and Intermediate Representation	10 hours
Parser: Role of parser, Top down Parsing-LL (1) parser, Bottom up parsing- shift reduce parser and LR (0), SLR parser, Introduction to Syntax directed Translation, Intermediate-Code Generation: Three-Address Code- Quadruples, Triples, Indirect triples Use Case-2: Role of CFG and parsing in Voice Assistant		
Module 5	Turing machine and Optimization	10 hours
Turing Machine: Basic Concept of Turing Machine, Variants of Turing Machine, Universal Turing Machine Code optimization and generation: Basic Block, Flow graph, DAG, Optimization Techniques		
Total Lecture Hours		48 hours
Textbook:		
S.No	Book Title	
1	K.L.P. Mishra, and N. Chandrasekharan , Theory of Computer Science-Automata, Languages and Computation	
2	Alfred V. Aho, Jeffrey D Ullman , Compilers: Principles, Techniques and Tools	

Branch Wise Applications

- It can compute man-made problems as well as natural phenomena.
- Automata theory has a lot of applications in real life as well, such that: Lambda calculus, Combinatory logic, Markov algorithm, and Register, Natural Language Processing ,Compiler Design and Lexical analysis and Semantic analysis. Also, the concept of Formal languages and automata theory can overlap with other subjects as well.

Course Objectives

- The primary objective of this course is to provide a foundational understanding of Automata Theory and its role in the language processing systems, also explores their application in fields like Natural Language Processing (NLP), speech recognition.

Module-3

Context Free Grammar and Push Down Automata

Context Free Grammar (CFG): Definition and Language, Derivations, Parse Trees and Ambiguity, Simplification of CFG, Push Down Automata: Definition of the Pushdown Automata, Languages of PDA

Course Outcome

Theory of Computation (BCSE0404X)

Course Outcome (CO)		Bloom's Knowledge Level (KL)
After completion of the course, the student will be able to:		
CO 1	Identify the fundamental concepts of automata theory, formal languages and compiler components.	K2
CO 2	Understand the role of finite automata, regular expressions, and grammar rules in language processing.	K2
CO 3	Demonstrate context-free grammars, pushdown automata, and syntax-directed translation to construct intermediate code for language processors.	K3
CO 4	Analyze various parsing strategies, code translation methods, and intermediate representations in compiler phases.	K4
CO 5	Analyze the functioning of Turing machine models and optimization techniques in code generation for performance improvement.	K4

Program Outcomes (POs)

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

Program Outcomes (POs)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

Program Outcomes (POs)

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

CO-PO Correlation Matrix

CO-PO Mapping	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	2	2	2	-	-	-	-	-	-	2
CO2	3	3	3	2	1	-	-	-	-	-	2
CO3	3	3	3	3	1	-	-	-	-	-	2
CO4	3	3	3	3	1	-	-	-	-	-	2
CO5	3	3	3	3	1	-	-	-	-	-	2
Average	3	2.8	2.8	2.6	1	-	-	-	-	-	2

Program Specific Outcomes

On successful completion of graduation degree, the Information Technology graduates will be able to:

PSO1	Work as a software engineer in different dimensions of information technology to provide solutions to real-world and industrial problems in an eco-friendly manner
PSO2	Apply knowledge of core subjects with multi-disciplinary and modern computing techniques to solve complex IT problems.
PSO3	Establish a successful and innovative career by practicing lifelong learning and communicating in a team or as an individual to manage projects with ethical concerns.

CO-PSO Correlation Matrix

CO-PSO Mapping	PSO1	PSO2	PSO3
CO1	2	3	1
CO2	2	3	1
CO3	3	3	2
CO4	3	3	2
CO5	3	2	2
Average	2.6	2.8	1.6

End Semester Question Paper Template

Printed Page:	Subject Code:
	Roll. No:
NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY, GREATER NOIDA	
(An Autonomous Institute Affiliated to AKTU, Lucknow)	
B.Tech	
SEM: - THEORY EXAMINATION (2025 - 2026)	
Subject:	
Time: 3 Hours	Max. Marks: 100
General Instructions: IMP: Verify that you have received the question paper with the correct course, code, branch etc. 1. This Question paper comprises of three Sections -A, B, & C. It consists of Multiple-Choice Questions (MCQ's) & Subjective type questions. 2. Maximum marks for each question are indicated on right -hand side of each question. 3. Illustrate your answers with neat sketches wherever necessary. 4. Assume suitable data if necessary. 5. Preferably, write the answers in sequential order. 6. No sheet should be left blank. Any written material after a blank sheet will not be evaluated/checked.	

End Semester Question Paper Templates

<u>SECTION-A</u>		20
1. Attempt all parts:-		
1-a.		1
1-b.		1
1-c.		1
1-d.		1
1-e.		1
1-f.		1
1-g.		1
1-h.		1
1-i.		1
1-j.		1
1-f.		1
2 Attempt all parts:-		
2.a.		2
2.b.		2
2.c.		2
2.d.		2
2.e.		2

End Semester Question Paper Templates

SECTION-B		30
3. Attempt all parts: -		
3.a. Answer any one of the following: -		
3.a.(i)		6
3.a.(ii)		6
3.b. Answer any one of the following: -		
3.b.(i)		6
3.b.(ii)		6
3.c. Answer any one of the following: -		
3.c.(i)		6
3.c.(ii)		6
3.d. Answer any one of the following: -		
3.d.(i)		6
3.d.(ii)		6
3.e. Answer any one of the following: -		
3.e.(i)		6
3.e.(ii)		6

End Semester Question Paper Templates

<u>SECTION-C</u>		50
4. Answer any <u>one</u> of the following: -		
4-a.		10
4-b.		10
5. Answer any <u>one</u> of the following: -		
5-a.		10
5-b.		10
6. Answer any <u>one</u> of the following: -		
6-a.		10
6-b.		10
7. Answer any <u>one</u> of the following: -		
7-a.		10
7-b.		10
8. Answer any <u>one</u> of the following: -		
8-a.		10
8-b.		10

Program Educational Objectives

PEO'S of the B. Tech. in Information Technology are:

The graduates will be:

PEO1: To apply sound knowledge combining technical expertise, and problem-solving skills to fulfill the needs of the IT industry and stay updated with the latest emerging trends.

PEO2: To provide sustainable solutions for real-life problems by designing innovative and interdisciplinary systems using the latest digital technologies for serving academia, industry, and society.

PEO3: To reflect professional attitude, social ethics, communication, teamwork, and leadership skills in research and product development to face global challenges.

PEO4: To pursue higher studies and develop their career successfully in the software industry and R&D sectors with a lifelong learning frame of mind.

.

Unit Objective

- To understand the definition of Context Free Grammar (CFG) and the types of languages generated by CFG.
- To explain the concept of derivations in CFG and analyze how strings are generated step by step.
- To study parse trees and identify ambiguity in grammars through multiple derivations.
- To learn the techniques for simplification of CFG by removing useless symbols, null productions, and unit productions.
- To understand the definition and structure of Pushdown Automata (PDA) as a computational model.
- To analyze the relationship between Pushdown Automata and Context Free Languages, and identify the languages accepted by PDA.

Topic Objective

Topic	Objectives
Context Free Grammar (CFG): Definition and Language	To understand the formal definition of Context Free Grammar and identify the class of languages generated by CFG.
Derivations in CFG	To learn leftmost and rightmost derivations and analyze the step-by-step generation of strings in CFG.
Parse Trees and Ambiguity	To construct parse trees and recognize ambiguous grammars through multiple parse trees or derivations.
Simplification of CFG	To understand and apply methods for simplifying CFG by eliminating useless symbols, null productions, and unit productions.
Pushdown Automata (PDA): Definition	To understand the structure, components, and working of Pushdown Automata as an abstract machine.
Languages of Pushdown Automata	To analyze the types of languages accepted by PDA and establish the equivalence between PDA and Context Free Languages.

Prerequisite:

- Basic knowledge of formal languages and automata concepts such as alphabets, strings, and languages.
- Understanding of regular languages, regular expressions, and finite automata (DFA/NFA).
- Familiarity with grammar basics, including terminals, non-terminals, production rules, and derivations.
- Basic understanding of set theory and mathematical notation used in language definitions.
- Awareness of language recognition models and their role in classifying languages.

Recap:

- Regular expressions are the way of representation of the FA.
- RE can be converted into FA.
- FA can be converted into regular expressions.
- Arden's Theorem is used to solve regular expression equations and obtain expressions from finite automata.
- Regular grammars generate regular languages and are classified as right-linear and left-linear grammars.
- The lexical analyzer is the first phase of a compiler and converts source code into tokens.
- Tokens are specified using regular expressions and recognized using finite automata.
- Lex is a tool used to automatically generate lexical analyzers based on token specifications.

A Grammar is a set of formal rules that define how to generate syntactically correct strings or sentences in a language.

A Grammar **G** can be formally written as a 4-tuple (N, T, S, P) where –

- **N** or V_N is a set of variables or non-terminal symbols.
- **T** or Σ is a set of Terminal symbols.
- **S** is a special variable called the Start symbol, $S \in N$
- **P** is Production rules for Terminals and Non-terminals. A production rule has the form $\alpha \rightarrow \beta$, where α and β are strings on $V_N \cup \Sigma$ and least one symbol of α belongs to V_N .

Examples of Grammar

Example 1

Grammar G1 –

$(\{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\})$

Here,

- **S**, **A**, and **B** are Non-terminal symbols;
- **a** and **b** are Terminal symbols
- **S** is the Start symbol, $S \in N$
- Productions,
- **P** : $S \rightarrow AB, A \rightarrow a, B \rightarrow b$

Example 2

Grammar G2 –

$((\{S, A\}, \{a, b\}, S, \{S \rightarrow aAb, aA \rightarrow aaAb, A \rightarrow \epsilon\}))$

Here,

- **S** and **A** are Non-terminal symbols.
- **a** and **b** are Terminal symbols.
- ϵ is an empty string.
- **S** is the Start symbol, $S \in N$
- Production
- **P** : $S \rightarrow aAb, aA \rightarrow aaAb, A \rightarrow \epsilon$

Types of Grammar

The different types of grammar –

Grammar	Language	Automata	Production rules
Type-0	Recursively enumerable	Turing machine	No restriction
Type-1	Context-Sensitive	Linear-bounded non-deterministic machine	$\alpha A \beta \rightarrow \alpha \gamma \beta$
Type-2	Context-Free	Non-deterministic push down automata	$A \rightarrow \gamma$
Type-3	Regular	Finite state automata	$A \rightarrow \alpha B$ $A \rightarrow \alpha$

Context-Free Grammar and Context-Free Language

- A Grammar $G=(N,T,P,S)$ is said to be Context-Free Grammar(CFG) if the production rules of G are of the form: $A \rightarrow \alpha$, Where $\alpha \in (NUT)^*$.
 - It is a formal grammar which is used to generate all possible patterns of strings in a given formal language.
 - Context-Free Grammar are very useful in lexical analysis, which is a part of Compiler.
 - The Language generated by G is denote as $L(G)$.The language L is called Context-Free Language(CFL) if $L(G)$ is for CFG.
- Or
- The Language Generated by Context-Free Grammar(CFG) is Called Context-Free Language(CFL).

Q1: Write CFG which generates palindrome for binary string.

Palindrome is a string which will be same while reading from LHS or RHS
example 101, 010.....

- $S \rightarrow 0S0/1S1$
- $S \rightarrow 0/1/\epsilon$

Generate 0101010

- $S \rightarrow 0S0$ [$S \rightarrow 0S0$]
- $S \rightarrow 01S10$ [$S \rightarrow 1S1$]
- $S \rightarrow 010S010$ [$S \rightarrow 0S0$]
- $S \rightarrow 0101010$ [$S \rightarrow 1$]

Q2: Write CFG for regular expression $r = 0^*1(0+1)^*$

- $r = 0^*1(0+1)^*$
- $\underbrace{0^* 1}_{A} \underbrace{(0+1)^*}_{B}$
- A B
- $S \rightarrow A1B$
- $A \rightarrow 0A/\epsilon$
- $B \rightarrow 0B/1B/\epsilon$

Q3: Write CFG for regular expression $r = (a+b)^*aa(a+b)^*$

- $S \rightarrow AaaA$
- $A \rightarrow aA/\epsilon$
- $A \rightarrow bA/\epsilon$

Q4: Set of all strings of length 2

- $R=(a+b)(a+b)$
- $S \rightarrow AA$
- $A \rightarrow a/b$

Q.5 Let $\Sigma = \{ (,) \}$ and let $L = \{ w \in \Sigma^* \mid w \text{ is a string of balanced parentheses} \}$

- $S \rightarrow (S)S \mid \epsilon$

Q.6 Let $\Sigma = \{ a, b \}$ and let $L = \{ w \in \Sigma^* \mid w \text{ is a palindrome} \}$

- $S \rightarrow \epsilon \mid a \mid b \mid aSa \mid bSb$

Q.7: Set of all strings of $L=(a+b)^*$

- $S \rightarrow aS/bS/\epsilon$

Q.8: Set of all strings of length at least 2

- $R = (a+b)(a+b)(a+b)^*$
- $S \rightarrow AAB$
- $A \rightarrow a/b$
- $B \rightarrow aB/bB/\epsilon$

Q.9: Set of all strings of length at most 2

- $R = (a+b+\epsilon)(a+b+\epsilon)$
- $S \rightarrow AA$
- $A \rightarrow a/b/\epsilon$

Q.10: For $R = a(a+b)^*b$

- $S \rightarrow aAb$
- $A \rightarrow aA/bA/\epsilon$

Q.11: Construct a CFG for the regular expression $(0+1)^*$

The CFG can be given by,

Production rule (P):

- $S \rightarrow 0S \mid 1S$
- $S \rightarrow \epsilon$

Q.12: Construct a CFG for a language $L = \{wcwR \mid \text{where } w \in (a, b)^*\}$.

- $S \rightarrow aSa$ rule 1
- $S \rightarrow bSb$ rule 2
- $S \rightarrow c$ rule 3

Example 1

If there is a grammar

$G: N = \{S, A, B\} \quad T = \{a, b\} \quad P = \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\}$

Here **S** produces **AB**, and we can replace **A** by **a**, and **B** by **b**. Here, the only accepted string is **ab**, i.e.,

$L(G) = \{ab\}$

Example 2

Suppose we have the following grammar –

$G: N = \{S, A, B\} \quad T = \{a, b\} \quad P = \{S \rightarrow AB, A \rightarrow aA|a, B \rightarrow bB|b\}$

The language generated by this grammar –

$L(G) = \{ab, a^2b, ab^2, a^2b^2, \dots\}$

$L(G) = \{a^m b^n \mid m \geq 1 \text{ and } n \geq 1\}$

The grammar rule $x \rightarrow y$ is called production rule and application of production rule is called derivation.

Derivation is a sequence of production rules. It is used to get the input string through these production rules. During parsing, we have to take two decisions.

These are as follows:

- We have to decide the non-terminal which is to be replaced.
- We have to decide the production rule by which the non-terminal will be replaced.

Left Most and Right Most derivations

A string can be derived in many ways . But we restrict ourselves to :

1. Rightmost Derivations
2. Leftmost Derivations

1. Left-most Derivation

- In the left most derivation, the input is scanned and replaced with the production rule from left to right. So, in left most derivatives we read the input string from left to right.

2. Right-most Derivation

- In the right most derivation, the input is scanned and replaced with the production rule from right to left. So, in right most derivatives we read the input string from right to left.

Left Most Derivations [Example]

For the grammar given below :

$$S \rightarrow A1B$$

$$A \rightarrow 0A | \epsilon$$

$$B \rightarrow 0B | 1B | \epsilon$$

Give Leftmost derivation and rightmost derivation of string 1001.

Solution

Leftmost derivation of 1001:

$$S \rightarrow A1B$$

$$S \rightarrow 1B \quad [A \rightarrow \epsilon]$$

$$\rightarrow 10B \quad [B \rightarrow 0B]$$

$$\rightarrow 100B \quad [B \rightarrow 0B]$$

$$\rightarrow 1001B \quad [B \rightarrow 1B]$$

$$\rightarrow 1001 \quad [B \rightarrow \epsilon]$$

Right Most derivations [Example]

Rightmost derivation of 1001:

$S \rightarrow A1B$

$S \rightarrow A10B \quad [B \rightarrow 0B]$

$\rightarrow A100B \quad [B \rightarrow 0B]$

$\rightarrow A1001B \quad [B \rightarrow 1B]$

$\rightarrow A1001 \quad [A \rightarrow \epsilon]$

$\rightarrow 1001 \quad [B \rightarrow \epsilon]$

Q. Give the LMD and RMD for the following

Production rules:

$S = S + S$

$S = S - S$

$S = a \mid b \mid c$

Input: $a - b + c$

Derivation Tree or Parse Tree

Derivation tree is a graphical representation for the derivation of the given production rules for a given CFG. It is the simple way to show how the derivation can be done to obtain some string from a given set of production rules. The derivation tree is also called a parse tree.

A parse tree contains the following properties:

- The root node is always a node indicating start symbols.
- The derivation is read from left to right.
- The leaf node is always terminal nodes.
- The interior nodes are always the non-terminal nodes.

Rules for Constructing a Parse tree

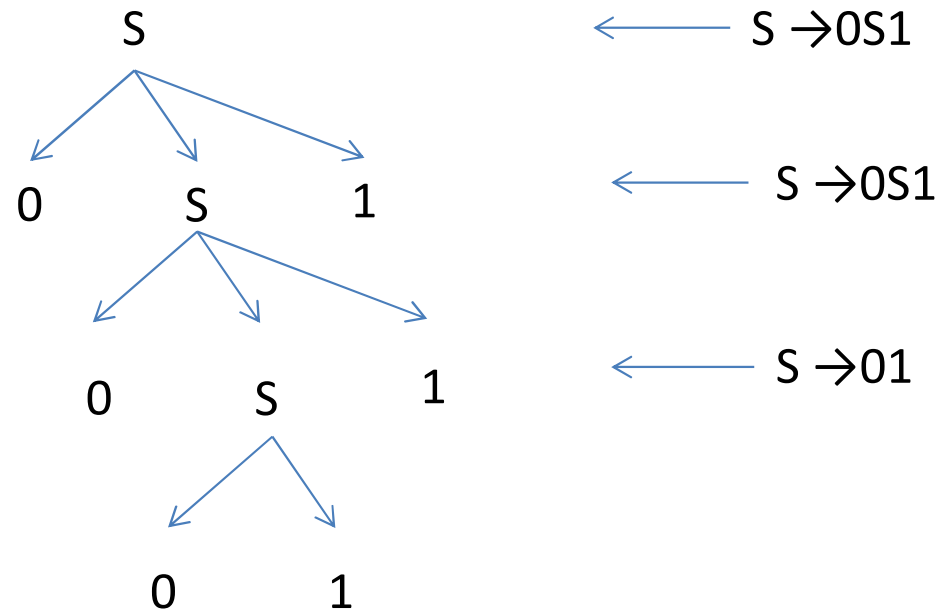
Following are the main rules for constructing a parse tree:

- First, place the initial symbol of the grammar as the root node of the parse tree.
- For each non-terminal symbol, create child nodes based on the grammar's production rules.
- Search for the appropriate production rule in the grammar that matches the non-terminal symbol being expanded.
- Repeat steps 2 and 3 for each non-terminal symbol until all non-terminal symbols have been expanded and only terminal symbols remain as leaf nodes.
- Finally verify the parse tree which ensures that the resulting parse tree conforms to the grammar rules and correctly represents the syntactic structure of the program.

Parse Tree [Example]

For a grammar given below $S \rightarrow 0S1 \mid 01$

- Give Parse tree derivation of 000111
- Derivation using parse tree:



A grammar is said to be ambiguous if there exists more than one leftmost derivation or more than one rightmost derivative or more than one Left Most parse tree or Right Most parse Tree for the given input string. If the grammar is not ambiguous then it is called unambiguous.

Definition:

$G = (N, T, P, S)$ is a CFG is said to be ambiguous if and only if there exist a string in T^* that has more than one parse tree.

where N is a finite set of variables.

T is a finite set of terminals.

P is a finite set of productions of the form, $A \rightarrow \alpha$, where A is a variable and $\alpha \in (N \cup T)^*$ S is a designated variable called the start symbol.

Ambiguous Grammar Example

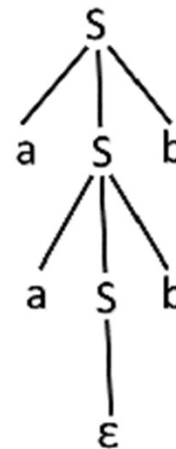
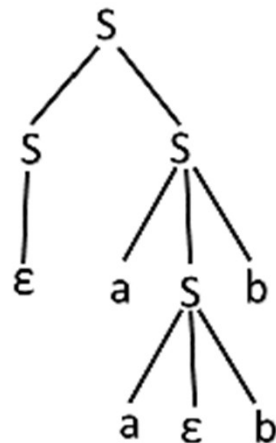
1. $S \rightarrow aSb \mid SS$

2. $S \rightarrow \epsilon$

Is this grammar ambiguous or not?

Solution:

For the string "aabb" the above grammar can generate two parse trees



Since there are two parse trees for a single string "aabb", the grammar G is ambiguous.

Ambiguous to Unambiguous Grammar Example

Ques. Show that the given grammar is ambiguous. Also, find an equivalent unambiguous grammar.

1. $E \rightarrow E + E$
2. $E \rightarrow E * E$
3. $E \rightarrow id$

Solution:

Let us derive the string "id + id * id"

As there are two different parse tree for deriving the same string, the given grammar is ambiguous.

Unambiguous grammar will be:

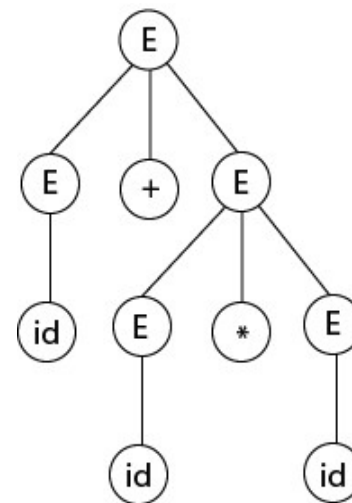
$E \rightarrow E + T$

$E \rightarrow T$

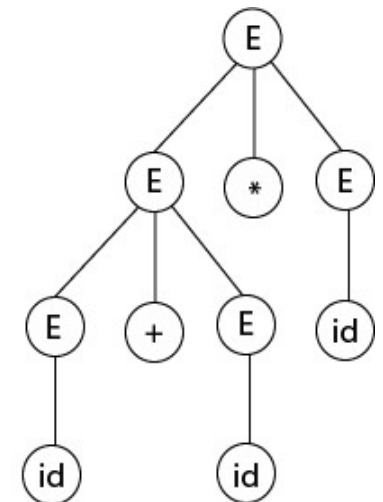
$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow id$



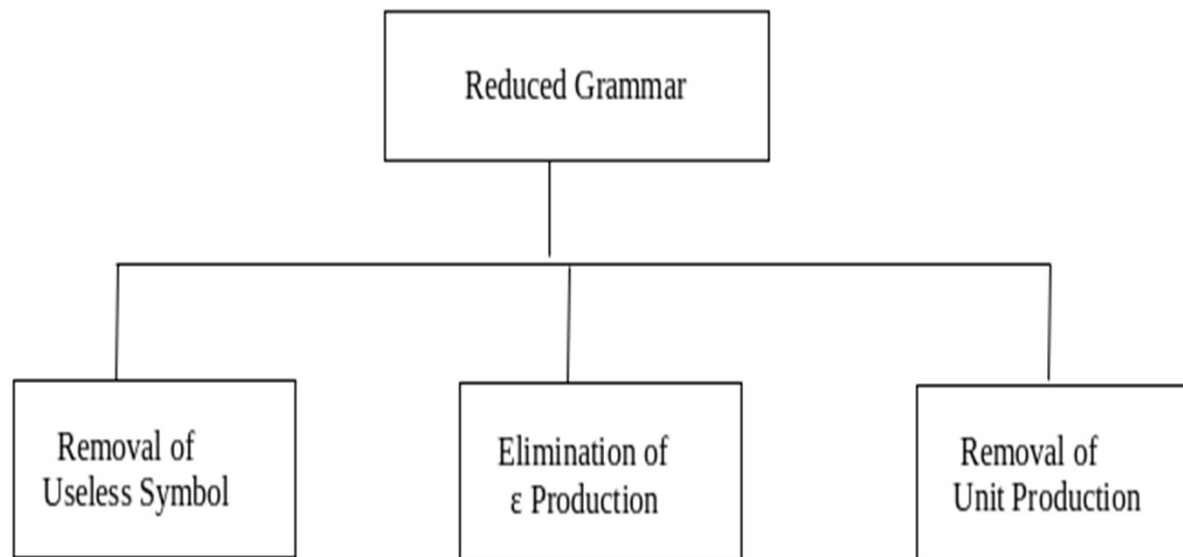
Parse tree 1



Parse tree 2

Simplification of CFG

- We must eliminate useless symbols, those variables or terminals that do not appears in any derivation of a terminal string from the start symbol.
- We must eliminate Null ϵ production , those of the form $X \rightarrow \epsilon$ for some variables X .
- We must eliminate unit production those of the form $X \rightarrow Y$ for variables X and Y .



Removal of Useless Symbols

- (a) Identified non generating symbols in given CFG and eliminate those productions which contains non generating symbols.**
- (b) Identified non reachable symbols in grammar and eliminate those productions which contain non reachable symbols.**

Example

$S \rightarrow aA | BC$

$A \rightarrow b | bB$

$B \rightarrow aB | \epsilon$

The final simplified grammar becomes –

$S \rightarrow aA$

$A \rightarrow b | bB$

$B \rightarrow aB | \epsilon$

Removal of ϵ Productions[Example]

- “If $A \rightarrow \epsilon$ is a production to be eliminate then we look for all productions, whose right side contain A and replace each occurrence of A in each of these productions to obtain the non ϵ productions.
- Now these resultant non ϵ production must be added to the grammar to keep the language generated, the same.”

Given Grammar:

- $S \rightarrow aA$
- $A \rightarrow b | \epsilon$

Solution:

Substitute ϵ on production $S \rightarrow aA$

$S \rightarrow a[\epsilon]$

$S \rightarrow a$

Then add this production into main one:

$S \rightarrow aA | a$

$A \rightarrow b$

Removal of Unit Productions

- Any production rule in the form $A \rightarrow B$ where $A, B \in \text{Non-terminal}$ is called **unit production**.

Removal Procedure –

- Step 1** – To remove $A \rightarrow B$, add production $A \rightarrow x$ to the grammar rule whenever $B \rightarrow x$ occurs in the grammar. [$x \in \text{Terminal}$, x can be Null]
- Step 2** – Delete $A \rightarrow B$ from the grammar.
- Step 3** – Repeat from step 1 until all unit productions are removed.

Removal of Unit Productions[Example]

Problem

Remove unit production from the following –

$$S \rightarrow XY,$$

$$X \rightarrow a,$$

$$Y \rightarrow Z \mid b,$$

$$Z \rightarrow M,$$

$$M \rightarrow N,$$

$$N \rightarrow a$$

Solution –

There are 3-unit productions in the grammar –

- $Y \rightarrow Z$, $Z \rightarrow M$, and $M \rightarrow N$

At first, we will remove $M \rightarrow N$.

- As $N \rightarrow a$, we add $M \rightarrow a$, and $M \rightarrow N$ is removed.

The production set becomes

- $S \rightarrow XY$, $X \rightarrow a$, $Y \rightarrow Z \mid b$, $Z \rightarrow M$, $M \rightarrow a$, $N \rightarrow a$

Now we will remove $Z \rightarrow M$.

- As $M \rightarrow a$, we add $Z \rightarrow a$, and $Z \rightarrow M$ is removed.

The production set becomes

- $S \rightarrow XY, X \rightarrow a, Y \rightarrow Z \mid b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$

Now we will remove $Y \rightarrow Z$.

- As $Z \rightarrow a$, we add $Y \rightarrow a$, and $Y \rightarrow Z$ is removed.

The production set becomes

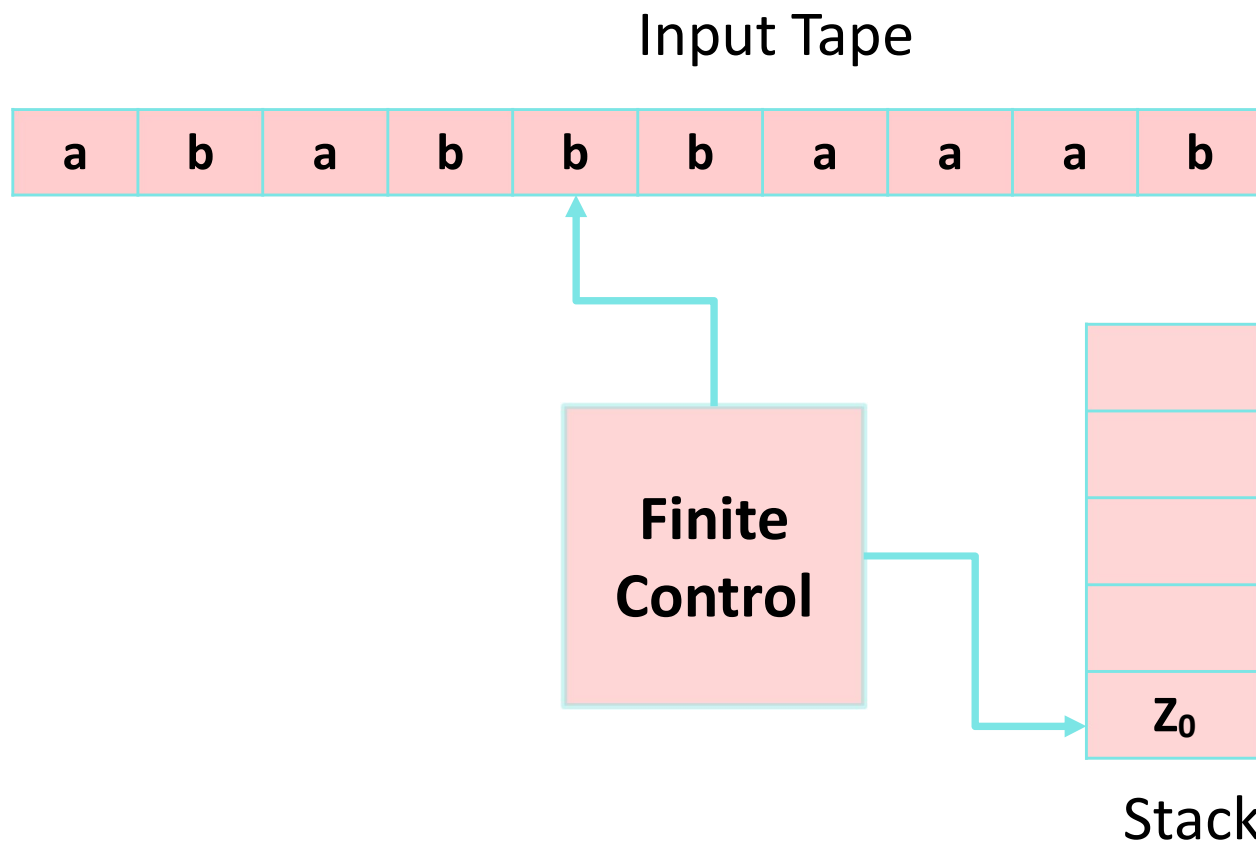
- $S \rightarrow XY, X \rightarrow a, Y \rightarrow a \mid b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$
- Now Z, M , and N are unreachable, hence we can remove those.

The final CFG is unit production free –

- $S \rightarrow XY, X \rightarrow a, Y \rightarrow a \mid b$

Pushdown Automata

- Informally a pushdown automata (PDA) is Finite Automata with control of both an input tape and a Stack to store what it has read.



Pushdown Automata

- A pushdown automaton (PDA) is a finite automaton equipped with a stack-based memory.
- Each transition
 - is based on the current input symbol and the top of the stack,
 - optionally pops the top of the stack, and
 - optionally pushes new symbols onto the stack.
- Initially, the stack holds a special symbol Z_0 that indicates the bottom of the stack.

Formal Definition of PDA

PDA is a machine with 7-tuples $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, where

- Q is a non empty finite set of states,
- Σ is an non empty finite set of input symbols,
- Γ is the stack alphabet of symbols that can be pushed on to the stack,
- $\delta : Q \times \{ \Sigma \cup \epsilon \} \times \Gamma^* \rightarrow (Q \times \Gamma^*)$ is the transition function,
- $q_0 \in Q$ is the start state,
- $Z_0 \in \Gamma$ is the stack start symbol, and
- $F \subseteq Q$ is the set of accepting states.

Instantaneous Description of a PDA

- The Instantaneous description is called as an informal notation, and explains how a Push down automata (PDA) computes the given input string and makes a decision that the given string is accepted or rejected.
- The PDA involves both state and content of the stack.
- Stack is often one of the important parts of PDA at any time.
- So, we make a convenient notation for describing the successive configurations of PDA for string processing.
- The factors of PDA notation by triple (q, w, γ) were
 - q is the current state.
 - w is the remaining input alphabet.
 - γ is the current contents of the PDA stack.
- Generally, the leftmost symbol indicates the top of the stack γ and the bottom at the right end. This type of triple notation is called an instantaneous description or ID of the pushdown automata.

Instantaneous Description of a PDA

- Write down the IDs or moves for input string $w = \text{“aabb”}$ of PDA as $M = (\{q_0, q_1, q_2\}, \{a, b\}, \{a, b, Z_0\}, \delta, q_0, Z_0, \{q_2\})$, where δ is defined by following rules:
 - $\delta(q_0, a, Z_0) = \{(q_1, aZ_0)\}$ Rule (1)
 - $\delta(q_1, a, a) = \{(q_1, aa)\}$ Rule (2)
 - $\delta(q_1, b, a) = \{(q_2, \lambda)\}$ Rule (3)
 - $\delta(q_2, b, a) = \{(q_2, \lambda)\}$ Rule (4)
 - $\delta(q_2, \lambda, Z_0) = \{(q_2, \lambda)\}$ Rule (5)
- Also check string w is accepted by PDA or not?

Instantaneous Description of a PDA

- **Solution:** Instantaneous Description for string $w = \text{“aabb”}$
- $(q_0, \text{aabb}, Z_0) \vdash (q_1, \text{abb}, \text{a}Z_0)$ as per Rule (1)
 $\vdash (q_1, \text{bb}, \text{aa}Z_0)$ as per Rule (2)
 $\vdash (q_2, \text{b}, \text{a}Z_0)$ as per Rule (3)
 $\vdash (q_2, \lambda, Z_0)$ as per Rule (3)
- Finally, PDA reached a configuration of (q_2, λ, Z_0) i.e. the input tape is empty or input string w is completed, PDA stack is empty and PDA has reached a final state. So the string ‘ w ’ is **accepted**.

Objective: To understand the language accepted by PDA.

A language of a PDA can be accepted by two ways:

- Through final State
- Through empty stack

It is also possible to convert between the two states:

- From Final State to empty stack
- From empty stack to final state

Acceptance by Empty Stack

- Let the PDA , $M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, \Phi)$ then the language accepted by an empty stack is given by:

$$L(M) = \left\{ w \mid (q_0, w, z_0) \xRightarrow[M]{*} (q_1, \varepsilon, \varepsilon) \right\}$$

Where q_1 is any state belonging to Q and the stack becomes empty on application of input string w .

Acceptance by Final State

- Let the PDA , $M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$ then the language accepted by M through a final state is given by:

$$L(M) = \left\{ w \mid (q_0, w, z_0) \xRightarrow[M]{*} (q_1, \epsilon, \alpha) \right\}$$

- Where state $q_1 \in F$, α , the final contents of the stack are irrelevant as a string is accepted through a final state.

PDA for $L = \{a^n b^n \mid n > 0\}$.

Acceptance by Final state

$$\delta(q_0, a, Z_0) = (q_1, aZ_0)$$

$$\delta(q_1, a, a) = (q_1, aa)$$

$$\delta(q_1, b, a) = (q_2, \epsilon)$$

$$\delta(q_2, b, a) = (q_2, \epsilon)$$

$$\delta(q_2, \epsilon, Z_0) = (q_f, Z_0)$$

$$M = \{\{q_0, q_1, q_f\}, \{a, b\}, \{a, Z_0\}, \delta, q_0, Z_0, q_f\}$$

PDA for $L = \{a^n b^n \mid n > 0\}$.

Acceptance by Empty stack

$$\delta(q_0, a, Z_0) = (q_1, aZ_0)$$

$$\delta(q_1, a, a) = (q_1, aa)$$

$$\delta(q_1, b, a) = (q_2, \epsilon)$$

$$\delta(q_2, b, a) = (q_2, \epsilon)$$

$$\delta(q_2, \epsilon, Z_0) = (q_2, \epsilon)$$

$$M = \{\{q_0, q_1\}, \{a, b\}, \{a, Z_0\}, \delta, q_0, Z_0, \phi\}$$

PDA(Example)

PDA for $L = \{a^n c^m b^n \mid m, n > 0\}$.

$$\delta(q_0, a, Z_0) = (q_1, aZ_0)$$

$$\delta(q_1, a, a) = (q_1, aa)$$

$$\delta(q_1, c, a) = (q_2, a)$$

$$\delta(q_2, c, a) = (q_2, a)$$

$$\delta(q_2, b, a) = (q_3, \epsilon)$$

$$\delta(q_3, b, a) = (q_3, \epsilon)$$

$$\delta(q_3, \epsilon, Z_0) = (q_f, Z_0)$$

$$M = \{\{q_0, q_1, q_2, q_f\}, \{a, b, c\}, \{a, Z_0\}, \delta, q_0, Z_0, q_f\}$$

PDA for $L = \{a^n c b^{2n} \mid n > 0\}$.

$$\delta(q_0, a, Z_0) = (q_1, aZ_0)$$

$$\delta(q_1, a, a) = (q_1, aa)$$

$$\delta(q_1, c, a) = (q_2, a)$$

$$\delta(q_2, b, a) = (q_3, a)$$

$$\delta(q_3, b, a) = (q_4, \epsilon)$$

$$\delta(q_4, b, a) = (q_3, a)$$

$$\delta(q_4, \epsilon, Z_0) = (q_f, Z_0)$$

$$M = \{\{q_0, q_1, q_2, q_f\}, \{a, b, c\}, \{a, Z_0\}, \delta, q_0, Z_0, q_f\}$$

Non-deterministic PDA

- Construct a PDA for $L = \{ww^R \mid w \in (0, 1)^*\}$.

**Why It is
Non-deterministic ??**

**Why not
deterministic ??**

- Construct a PDA for $L = \{wcw^R \mid w \in (0, 1)^*\}$.

Difference between DPDA and NPDA

DPDA	NPDA
In NPDA, there may exits exactly one transition for each input symbol.	In PDA, there may exits more than one transition for each input symbol
Table contains single entities	Table may contain multiple defined entities.
Less powerful than NPDA.	More powerful than DPDA.
Ex: PDA for $L = \{wcw^R \mid w \in (0, 1)^*\}$	Ex: PDA for $L = \{ww^R \mid w \in (0, 1)^*\}$

Youtube/other Video Links

- <https://www.youtube.com/watch?v=6b40kKe2SFg>
- <https://www.youtube.com/watch?v=-aIRqNnUvEg&list=PL85CF9F4A047C7BF7>
- <https://www.youtube.com/watch?v=Xk-MTgB68rQ>

1. Write the formal definition of a Context Free Grammar (CFG) using its components.
2. State the meaning of language generated by a grammar and represent it symbolically.
3. Distinguish between leftmost derivation and rightmost derivation in CFG.
4. For the grammar: $S \rightarrow aSb \mid ab$, show a valid derivation for the string aabb.
5. Illustrate a parse tree for a given string derived from a CFG.
6. State the condition under which a context free grammar becomes ambiguous.
7. Give one example of an ambiguous grammar along with a string that proves ambiguity.
8. List any two steps involved in simplification of a CFG.
9. Write the formal definition of Push Down Automata (PDA) with its tuple representation.
10. State the relationship between Push Down Automata and Context Free Languages.

Weekly Assignment

1. Let G be the grammar $S \rightarrow aB \mid bA$, $A \rightarrow a \mid aS \mid bAA$, $B \rightarrow b \mid bS \mid aBB$. For the string "aaabbabbba", find
 - a. Parse tree
 - b. Leftmost derivation
 - c. Rightmost derivation
 - d. Is the grammar unambiguous?

CO3 K3
2. Write CFG for regular expression $r = (a+b)^*aa(a+b)^*$.

CO3 K3
3. Let $G = (\{S, C\}, \{a, b\}, P, S)$ where P consists of $S \rightarrow aCa$, $C \rightarrow aCa \mid b$. Find $L(G)$.

CO3 K3
4. Simplified the following Context Free Grammar (CFG): $S \rightarrow ABA$, $A \rightarrow 0A \mid \epsilon$, $B \rightarrow 1B \mid \epsilon$.

CO3 K3
5. Construct a PDA for the language $L = \{a^n b^n \mid n \geq 1\}$.

CO3 K5

1. A Context Free Grammar is defined as

- A) (Q, Σ, δ, q_0)
- B) (V, T, P, S)
- C) $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$
- D) (Σ, δ, q_0)

✓ **Answer: B**

2. The language generated by a grammar G is represented as

- A) Σ^*
- B) $L(G)$
- C) $P(G)$
- D) $T(G)$

✓ **Answer: B**

3. A leftmost derivation replaces

- A) Any non-terminal
- B) Rightmost non-terminal first
- C) Leftmost non-terminal first
- D) Terminal symbols only

✓ **Answer: C**

4. A grammar is ambiguous when

- A) Multiple terminals exist
- B) More than one start symbol exists
- C) A string has more than one parse tree
- D) The grammar contains ϵ -productions

✓ **Answer: C**

5. The grammar

$$S \rightarrow S + S \mid S * S \mid id$$

is classified as

- A) Regular grammar
- B) Unambiguous grammar
- C) Ambiguous grammar
- D) Linear grammar

✓ **Answer: C**

6. The process of removing useless symbols is part of

- A) Grammar derivation
- B) Grammar ambiguity
- C) Grammar simplification
- D) Grammar validation

✓ **Answer: C**

7. A Push Down Automaton is formally defined as

- A) 4-tuple
- B) 5-tuple
- C) 6-tuple
- D) 7-tuple

✓ **Answer: D**

8. Push Down Automata recognize

- A) Regular languages
- B) Context free languages
- C) Recursive languages
- D) Context sensitive languages

✓ **Answer: B**

9. The stack in PDA is mainly used for

- A) Input storage
- B) Output generation
- C) Handling nested structures
- D) State transition storage

✓ **Answer: C**

10. Acceptance of language by PDA can be done by

- A) Final state only
- B) Empty stack only
- C) Both final state and empty stack
- D) Start state only

 **Answer: C**

Expected Questions for University Exam

1. Let $G = (\{S, C\}, \{a, b\}, P, S)$ where P consists of $S \rightarrow aCa$, $C \rightarrow aCa \mid b$. Find $L(G)$. CO3 K3
2. Find $L(G)$, where $G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S1, S \rightarrow \epsilon\}, S)$ CO3 K3
3. Find out the context free language CO3 K3
 - $S \rightarrow aSb \mid aAb$
 - $A \rightarrow bAa$
 - $A \rightarrow ba$
4. Give the CFG of the following language: $0(0+1)^*01(0+1)^*1$ CO3 K3
5. Give CFG for matching parenthesis. CO3 K3
6. Construct a PDA from the following CFG, $G = (\{S, X\}, \{a, b\}, P, S)$ where the productions are : $S \rightarrow XS \mid \epsilon$, $A \rightarrow aXb \mid Ab \mid ab$ CO3 K5
7. Construct a PDA that accepts $L = \{ ww^R \mid w = (a+b)^* \}$ CO3 K5
8. Construct a PDA that accepts $L = \{ wcw^R \mid w = (a+b)^* \}$ CO3 K5
9. Construct a PDA for the language $L = \{ a^n b^n c^m \mid n, m \geq 1 \}$ CO3 K5
10. Construct a PDA for the language $L = \{ a^n b^n \mid n \geq 1 \}$ CO3 K5

Printed Page:- 05

Subject Code:- ACSE0404/ACSEH0404

Roll. No:

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY, GREATER NOIDA

(An Autonomous Institute Affiliated to AKTU, Lucknow)

B.Tech

SEM: IV - THEORY EXAMINATION (2022-2023)

Subject: Theory of Automata and Formal Languages

Time: 3 Hours

Max. Marks: 100

General Instructions:

IMP: Verify that you have received the question paper with the correct course, code, branch etc.

1. This Question paper comprises of three Sections -A, B, & C. It consists of Multiple Choice Questions (MCQ's) & Subjective type questions.

2. Maximum marks for each question are indicated on right-hand side of each question.

3. Illustrate your answers with neat sketches wherever necessary.

4. Assume suitable data if necessary.

5. Preferably, write the answers in sequential order.

6. No sheet should be left blank. Any written material after a blank sheet will not be evaluated/checked.

SECTION A

20

1. Attempt all parts:-

- 1-a. Can a DFA simulate NFA? (CO1) 1
- (a) NO
- (b) YES
- (c) Sometimes
- (d) Depends on NFA
- 1-b. Finite State Machine can recognize (CO1) 1
- (a) Any Grammar
- (b) Only CFG
- (c) Any unambiguous Grammar
- (d) Only Regular Grammar
- 1-c. Consider the following regular expression : (CO2) 1
- $R = (ab + abb)^* bbab$
- which of the following is not in R.
- (a) ababab

Page 1 of 5

- (b) ababbabbbab
- (c) ababbbbab
- (d) abbabbbab
- 1-d. The regular expression $(a + b)^*$ denotes all strings (CO2) 1
- (a) with zero or more instances of a and b both simultaneously
- (b) with one or more instances of a and b
- (c) any combination of a's and b's including null string.
- (d) None of these
- 1-e. Every CFG can be transferred into equivalent : (CO3) 1
- (a) Greiback Normal Form
- (b) CNF
- (c) Either GNF or CNF
- (d) All of mention
- 1-f. CFG is not closed under : (CO3) 1
- (a) Union
- (b) Kleene Star
- (c) Complementation
- (d) Product
- 1-g. A push down automata can represented using: (CO4) 1
- (a) Transition graph
- (b) Transition table
- (c) ID
- (d) All of the mentioned
- 1-h. The transition a Push down automaton makes is additionally dependent upon the: (CO4) 1
- (a) State
- (b) Unconsumed input
- (c) Stack content
- (d) All of the mentioned
- 1-i. Turing Machine consist of : (CO5) 1
- (a) Input Tape
- (b) Blank Symbol
- (c) Tape head

Page 2 of 5

- (d) All of these
- 1-j. A turing machine that is able to simulate other turing machines: (CO5) 1
- (a) Nested Turing Machine
(b) Universal Turing Machine
(c) Counter Turing Machine
(d) None of the mention

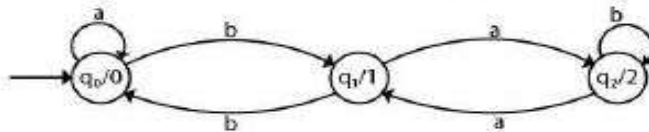
2. Attempt all parts:-

- 2.a. Explain the term Alphabets, Strings, Language in Finite Automata. (CO1) 2
- 2.b. Describe the definition of Regular Expression. (CO2) 2
- 2.c. Explain the Mathematical description of Context Free Grammar. (CO3) 2
- 2.d. Describe the explanation for the Moves of Pushdown Automata. (CO4) 2
- 2.e. Explain Church's Thesis. (CO5) 2

SECTION B

3. Answer any five of the following:-

- 3-a. Design Finite Automata that accepts set of strings containing exactly four 1's in every string over alphabet {0,1}. (CO1) 6
- 3-b. Convert the given Moore machine into its equivalent Mealy machine. (CO1) 6

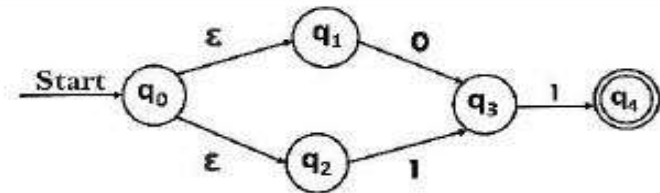


- 3-c. Write the regular expression for the language $L = \{ a^n b^m : (n+m) \text{ is even} \}$ (CO2) 6
- 3-d. Prove that $L = \{ a^n b^n a b^{n+1} \text{ for } n = 1, 2, 3, \dots \}$ is not regular. (CO2) 6
- 3.e. If CFG $\{G\}$ is $S \rightarrow SbS / a$, Show that G is ambiguous. (CO3) 6
- 3.f. Design a PDA for the language $L = \{ 0^n 1^{2n} : n > 0 \}$ (CO4) 6
- 3.g. Design a Turing Machine which recognize the language of Regular Expression (01^*0) . (CO5) 6

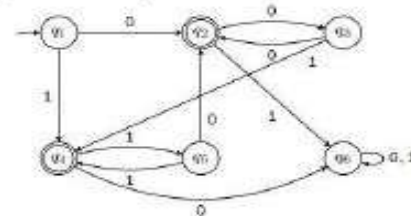
SECTION C

4. Answer any one of the following:-

- 4-a. Convert epsilon-NFA to NFA without epsilon. (CO1) 10

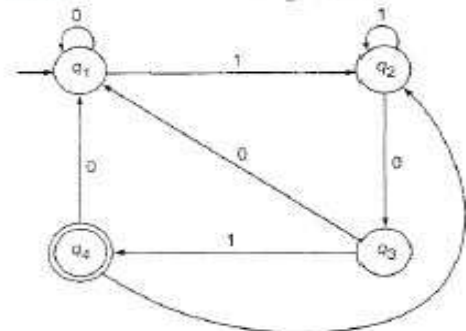


- 4-b. For the following DFA, determine the classes of equivalent states, and use them to provide the equivalent minimum-state DFA. (CO1) 10



5. Answer any one of the following:-

- 5-a. Write down the statement of Pumping Lemma for Regular Languages. also Describe the Closure Properties of Regular Languages. (CO2) 10
- 5-b. Prove Ardens's Theorem. Find the regular expression corresponding to Fig. (CO2) 10



6. Answer any one of the following:-

- 6-a. Consider the grammar (CO3) 10
- $S \rightarrow aB / bA$

- $A \rightarrow aS / bAA / a$
 $B \rightarrow bS / aBB / b$
 For the string aaabbabbba, find
 (i) The left most derivation and left most derivation tree
 (ii) The right most derivation and right most derivation tree
- 6-b. Describe the following : (CO3) 10
- (i) Eliminating the Use Less Symbols in CFG
 (ii) Removal of Unit Production in CFG
 (iii) Removal of Null - Production in CFG
 Find the Reduced Grammar that is equivalent to the CFG given below :
 $S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow C / b$
 $C \rightarrow D$
 $D \rightarrow E$
 $E \rightarrow a$
7. Answer any one of the following:-
- 7-a. Describe the Definition of Pushdown Automata. Is PDA more powerful than Finite Automata? if Yes then why? also Design PDA for Language $L = \{ a^m + b^n c^m / m, n > 1 \}$ (CO4) 10
- 7-b. Design a PDA for the following CFG : (CO4) 10
- $S \rightarrow \epsilon$
 $S \rightarrow SS$
 $S \rightarrow (S)$
 Show the transition relation on string () () for constructed PDA.
8. Answer any one of the following:-
- 8-a. Explain Instantaneous description of Turing Machine. Design the Turing Machine for : (CO5) 10
- (i) 1's Complement of any string
 (ii) 2's Complement of any string
- 8-b. Explain any two of the following : (CO5) 10
- (i) Universal Turing Machine
 (ii) Recursively Enumerable Language
 (iii) Halting Problem
 (iv) Post's Correspondence Problem

- Introduces Context Free Grammar (CFG) and its role in defining context-free languages.
- Explains derivations in CFG, including leftmost and rightmost derivations.
- Describes parse trees as a graphical representation of grammar derivations.
- Discusses ambiguity in grammars and how a string can have multiple parse trees.
- Covers methods for simplification of CFG by removing null productions, unit productions, and useless symbols.
- Introduces Pushdown Automata (PDA) as a language recognition model with stack memory.
- Explains the structure and operation of PDA.
- Establishes the relationship between PDA and context-free languages, showing that PDAs accept exactly the languages generated by CFGs.

1. K.L.P. Mishra, and N. Chandrasekharan , Theory of Computer Science- Automata, Languages and Computation
2. Alfred V. Aho, Jeffrey D Ullman , Compilers: Principles, Techniques and Tools
3. J Martin , Introduction to languages and the theory of computation
4. Allen I. Holub , Compiler Design in C

Thank You